Linear Regression

Open slides

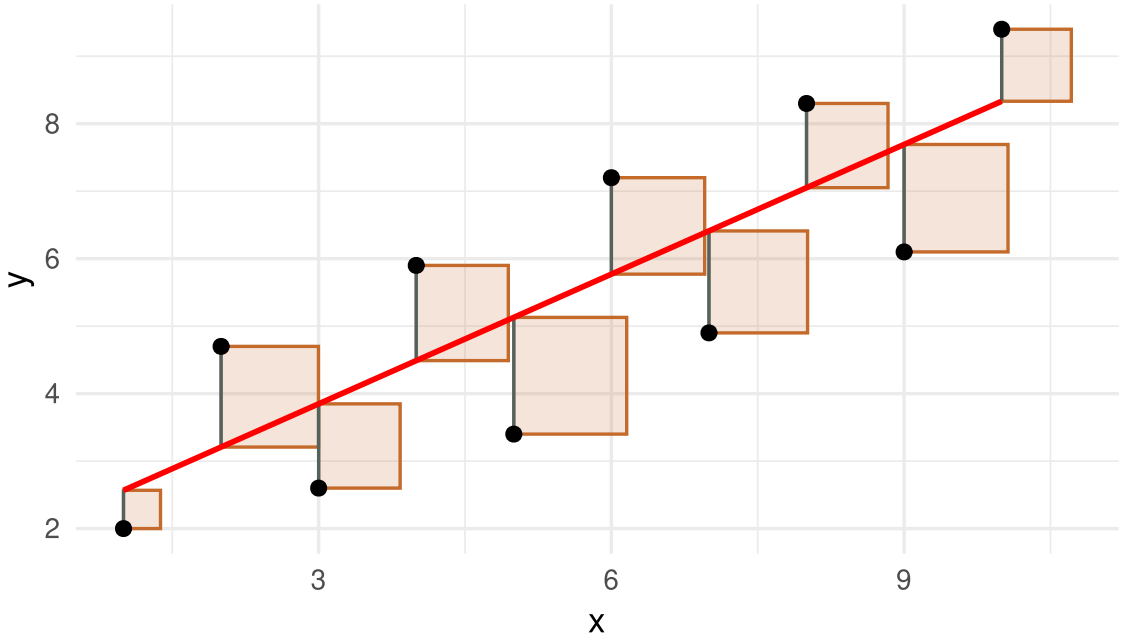
Linear Regression

Linear Regression deals with fitting predicted values based on input features and regression weights to observations. In ordinary least squares, we aim to find regression weights that minimize the mean squared error between predicted and observed values:

$$\hat{\beta} = argmin \sum_{i=1}^{n} (y_i - x_i \beta_i)^2$$

In the regression plots below, the vertical segments show residuals $e_i = y_i - \hat{y}_i$. A small toy plot first isolates what the "squares" in least squares mean.

```
toy_dat = data.frame(
  x = 1:10,
  y = c(2.0, 4.7, 2.6, 5.9, 3.4, 7.2, 4.9, 8.3, 6.1, 9.4)
)

toy_fit = lm(y ~ x, data = toy_dat)
toy_dat$pred = predict(toy_fit)

toy_squares = residual_square_data(toy_dat, "x", "y", "pred", every = 1, scale
= 0.55)

ggplot(toy_dat, aes(x = x, y = y)) +
  geom_rect(
    data = toy_squares,
    aes(xmin = xmin, xmax = xmax, ymin = ymin, ymax = ymax),
    inherit.aes = FALSE,
    fill = "#c46a2b",
    color = "#c46a2b",
    alpha = 0.18
  ) +
  geom_segment(aes(xend = x, yend = pred), color = "#155f7a", alpha = 0.65) +
  geom_point(size = 2.2) +
  geom_line(aes(y = pred), color = "red", linewidth = 1) +
  labs(x = "x", y = "y")
```



The vector x contains a "feature" or regression variable that establishes a connection with the observations. We can formulate most regression problems in terms of feature matrices and their estimated weights with respect to predicting the observations.

Feature / Design Matrix

A Feature or Design Matrix maps observations to individual feature criteria. The most fundamental feature matrix is a column vector with all 1s, meaning every observation has the same expression for that feature. In that case, the feature serves as a mapping to the overall mean of the observations.

We already know how to estimate the mean of y under the assumption of an underlying normal distribution. Now we estimate the mean with a linear regression ("Ordinary Least Squares" = OLS):

```
# load example data
data(airquality)

# our response vector is Wind
y = airquality$Wind

# create a design matrix for the mean
X = matrix(data = 1, nrow = length(y), ncol = 1)
colnames(X) = "Intercept"

str(X)
```

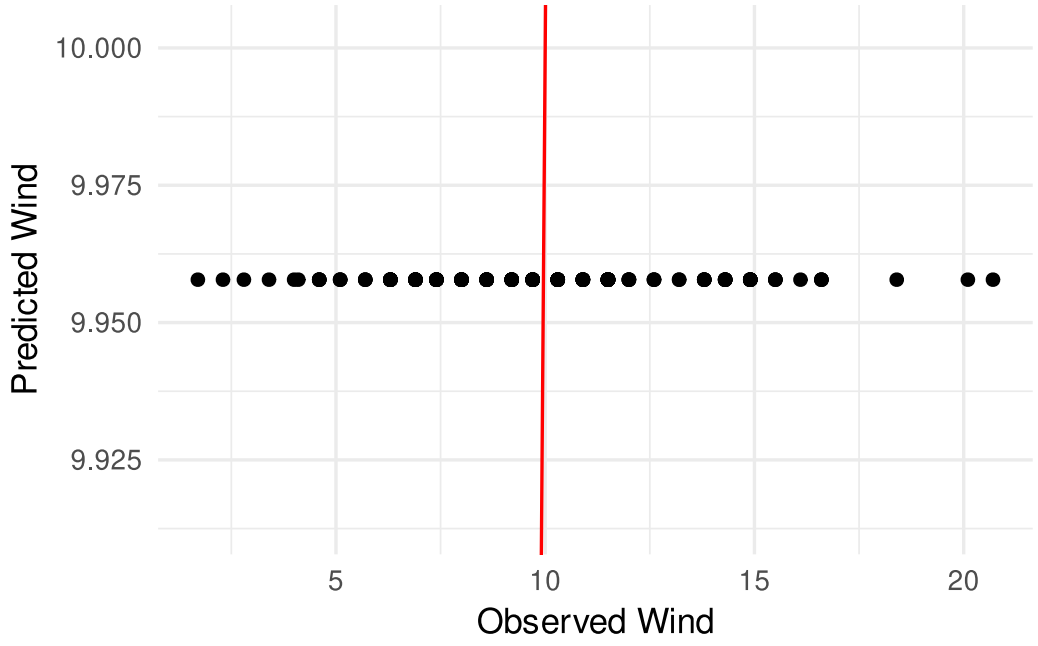
```
 num [1:153, 1] 1 1 1 1 1 1 1 1 1 1 ...
 - attr(*, "dimnames")=List of 2
  ..$ : NULL
  ..$ : chr "Intercept"
```

```
# define function to minimize the mean squared error between predicted and observed value
mse <- function(par, y, X) {

  # the weights are stored in "par", the length of that vector depends
  # on the number of columns in X (number of features)

  # predicted value
  pred = X %*% par

  # mean squared error
  mse = mean((y - pred)**2)

  return(mse)

}

p <- rep(0, ncol(X))
names(p) <- colnames(X)

res <- optim(
       par = p,
       fn = mse,
       y = y,
       X = X
            )

# our solution
res$par
```

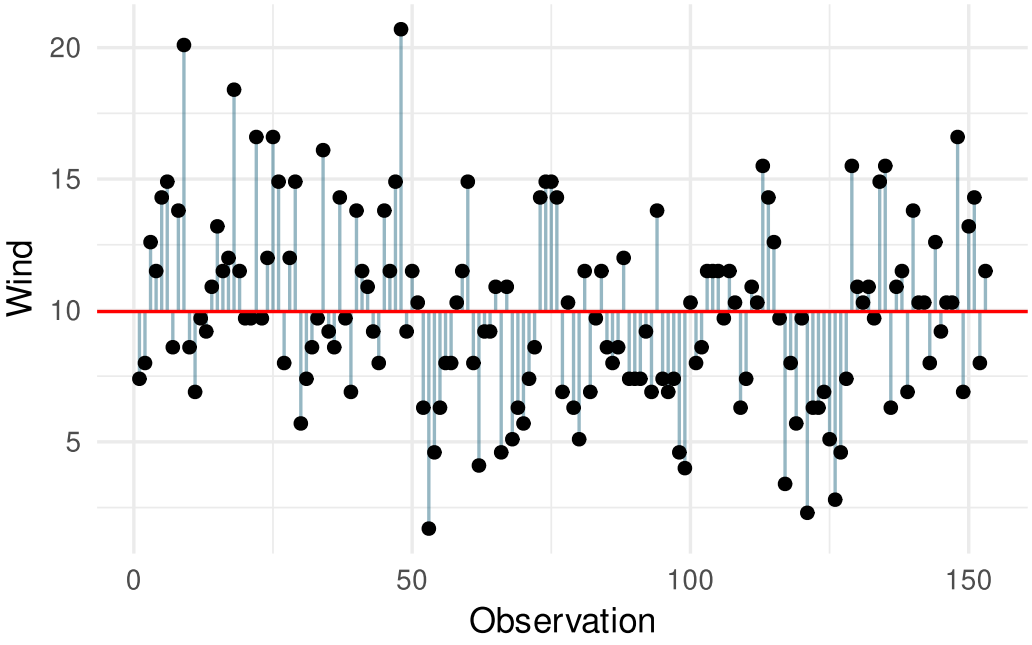
```
Intercept
 9.957812
```

```
# plot predicted vs observed
pred = X %*% res$par
```

```r
dat = data.frame(
     y = y,
     pred = as.numeric(pred),
     id = seq_along(y)
     )

ggplot(dat, aes(x = y, y = pred)) +
 geom_point() +
 geom_abline(intercept = 0, slope = 1, color = "red") +
 labs(x = "Observed Wind", y = "Predicted Wind")
```




```r
ggplot(dat, aes(x = id, y = y)) +
  geom_segment(aes(xend = id, yend = pred), color = "#155f7a", alpha = 0.45) +
  geom_point() +
  geom_hline(yintercept = res$par, color = "red") +
  labs(x = "Observation", y = "Wind")
```

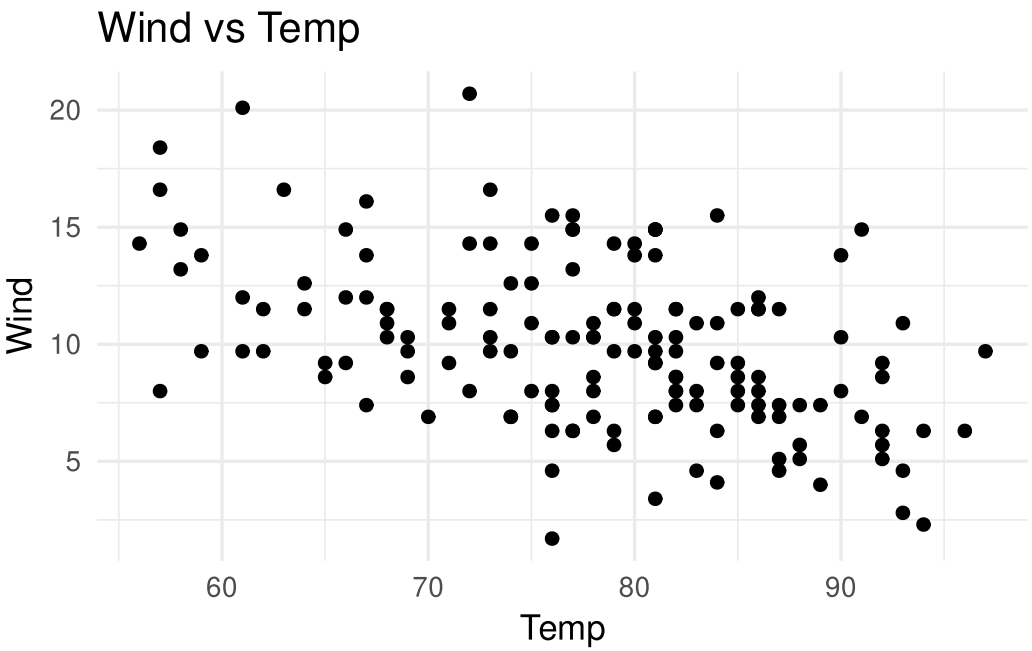


Adding Features to the Design Matrix: Multiple Linear Regression

When there is more than one feature in the design matrix we call that Multiple Linear Regression. The overall principle of the problem remains the same: We aim to minimize the mean squared error of our predictions.

```
# load example data
data(airquality)

# our response vector is Wind
y = airquality$Wind

# now we also want another feature: Temp
temp = airquality$Temp

# create a design matrix for the mean
X = matrix(data = 1, nrow = length(y), ncol = 1)

# add temperature as a feature to our design matrix
X = cbind(
   X,
   temp
   )

colnames(X) = c("Intercept", "Temp")
```

```r
# plot Wind vs Temp
ggplot(airquality, aes(x = Temp, y = Wind)) +
  geom_point() +
  labs(x = "Temp", y = "Wind", title = "Wind vs Temp")
```

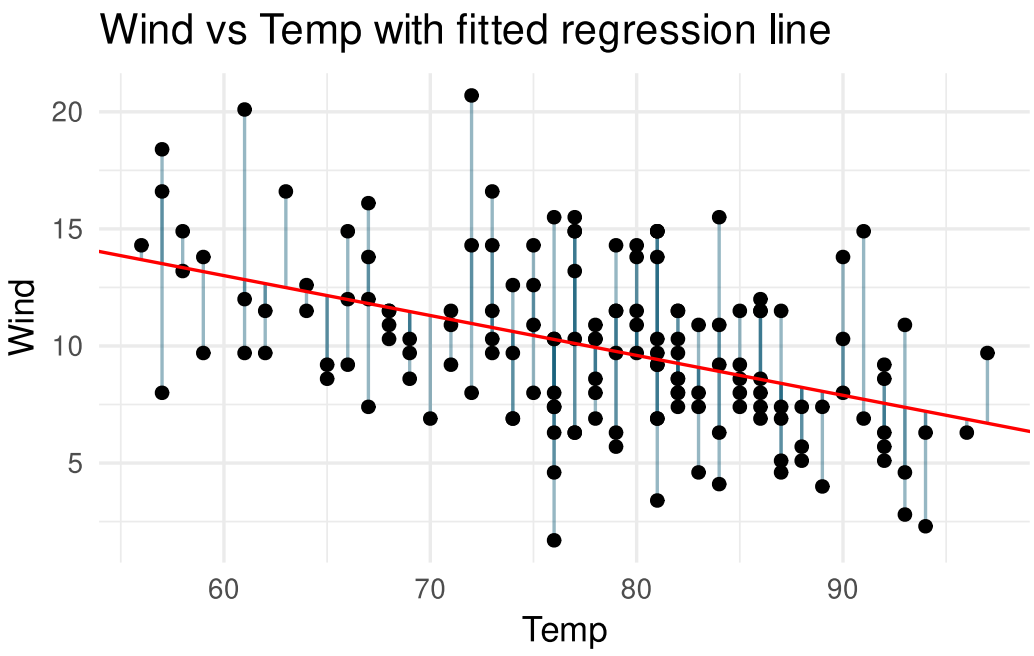



```r
# we can use the same function as before for finding the optimum feature weights
mse <- function(par, y, X) {

  # the weights are stored in "par", the length of that vector depends
  # on the number of columns in X (number of features)

  # predicted value
  pred = X %*% par

  # mean squared error
  mse = mean((y - pred)**2)

  return(mse)

}

p <- rep(0, ncol(X))
```

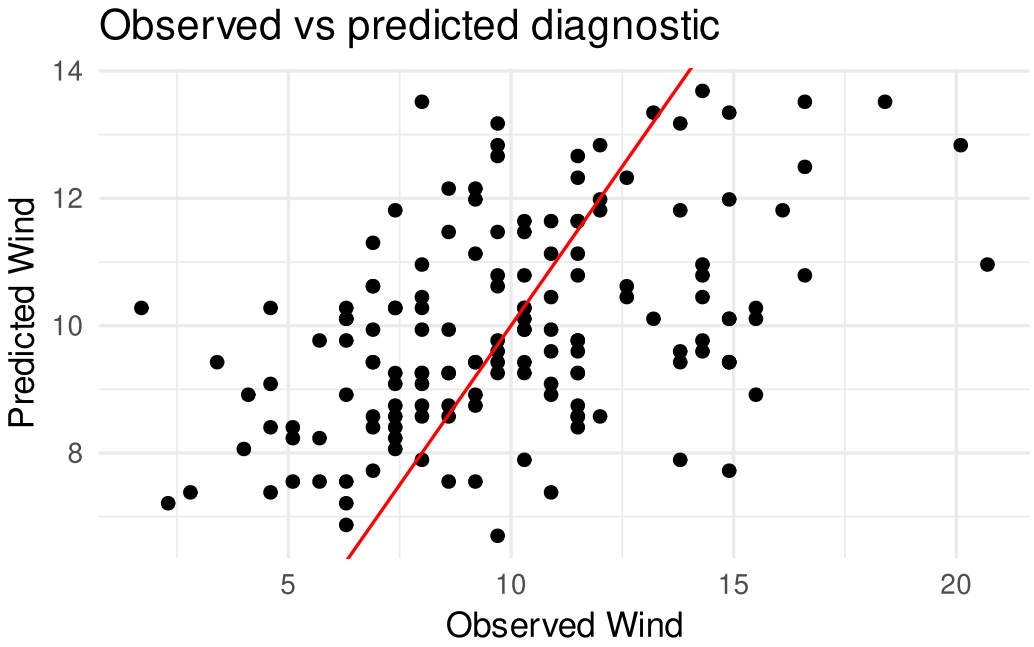
```r
names(p) <- colnames(X)

res <- optim(
       par = p,
       fn = mse,
       y = y,
       X = X
            )

# our solution
res$par
```

```
 Intercept       Temp
23.2314500 -0.1704349
```

```r
# plot predicted vs observed
pred = X %*% res$par

dat = data.frame(
     Wind = y,
     Temp = airquality$Temp,
     pred = as.numeric(pred),
     id = 1:length(y)
     )

ggplot(dat, aes(x = Temp, y = Wind)) +
  geom_segment(aes(xend = Temp, yend = pred), color = "#155f7a", alpha = 0.45) +
  geom_point() +
  geom_abline(intercept = res$par[1], slope = res$par[2], color = "red") +
  labs(x = "Temp", y = "Wind", title = "Wind vs Temp with fitted regression line")
```




```
ggplot(dat, aes(x = Wind, y = pred)) +
 geom_point() +
 geom_abline(intercept = 0, slope = 1, color = "red") +
 labs(x = "Observed Wind", y = "Predicted Wind", title = "Observed vs predicted
diagnostic")
```



Ridge Regression

$$\min_{\beta}\left(\sum_{i=1}^{n}\left(y_i-\beta_0-\sum_{j=1}^{p}\beta_j x_{ij}\right)^2+\lambda\sum_{j=1}^{p}\beta_j^2\right)$$

The ridge parameter λ controls how much we penalize large marker effects. We can visualize the solution space by evaluating the ridge criterion on a grid of λ values.

For a fixed λ, ridge regression still has a closed-form solution. If we write the design matrix as $X=[\mathbf{1},M]$ and do not penalize the intercept, then

$$\hat{\beta}_\lambda=(X'X+\lambda P)^{-1}X'y,\qquad P=\begin{bmatrix}0 & 0\\ 0 & I_p\end{bmatrix}.$$

In the code below, the marker matrix and response are centered first, so this reduces to

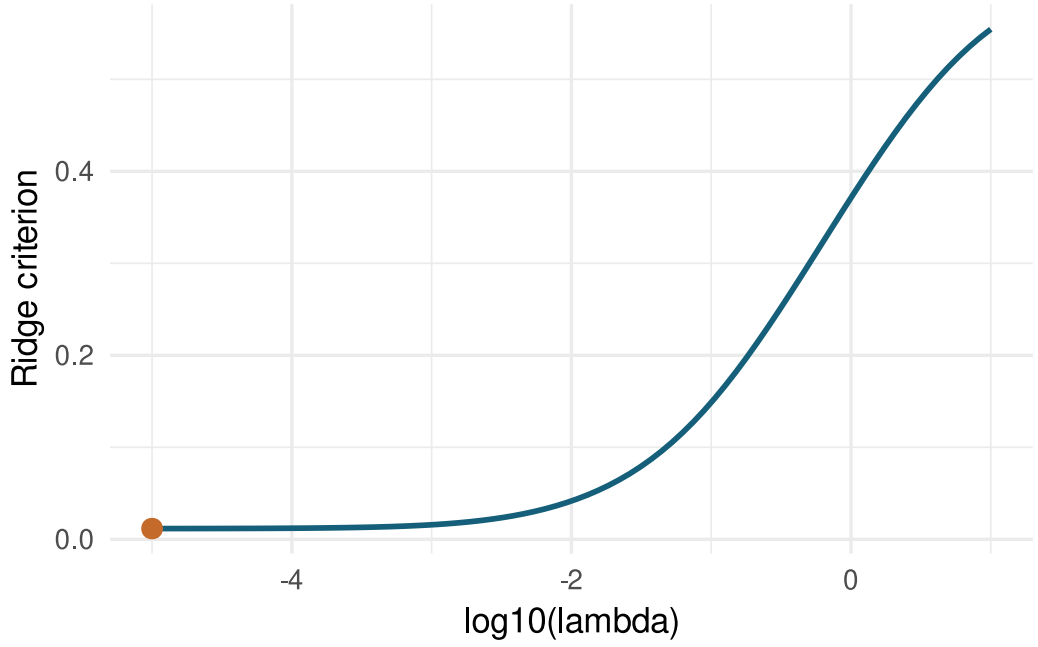
$$\hat{u}_\lambda=\left(\frac{M_c'M_c}{n}+\lambda I_p\right)^{-1}\frac{M_c'y_c}{n},\qquad \hat{\beta}_0=\bar{y}.$$

```
library(BGLR)

data(wheat)

subset = 1:100

y = wheat.Y[subset,2]
M = wheat.X[subset,1:300]

M_centered = scale(M, center = TRUE, scale = FALSE)
y_centered = y - mean(y)
n_obs = length(y)

fit_ridge_lambda <- function(lambda, y_centered, M_centered) {

  # closed-form ridge solution for a fixed lambda
  u = solve(
    crossprod(M_centered) / n_obs + diag(lambda, ncol(M_centered)),
    crossprod(M_centered, y_centered) / n_obs
  )

  pred = mean(y) + M_centered %*% u
  mse = mean((y - pred)**2)
  penalty = lambda * sum(u**2)
```

```
  data.frame(
    lambda = lambda,
    mse = mse,
    penalty = penalty,
    criteria = mse + penalty,
    sd_marker_effect = sd(as.numeric(u))
  )

}

lambda_grid = 10^seq(-5, 1, length.out = 80)

ridge_path = do.call(
  rbind,
   lapply(lambda_grid, fit_ridge_lambda, y_centered = y_centered, M_centered =
M_centered)
)

best_lambda = ridge_path[which.min(ridge_path$criteria), ]

kable(best_lambda, digits = 4)
```

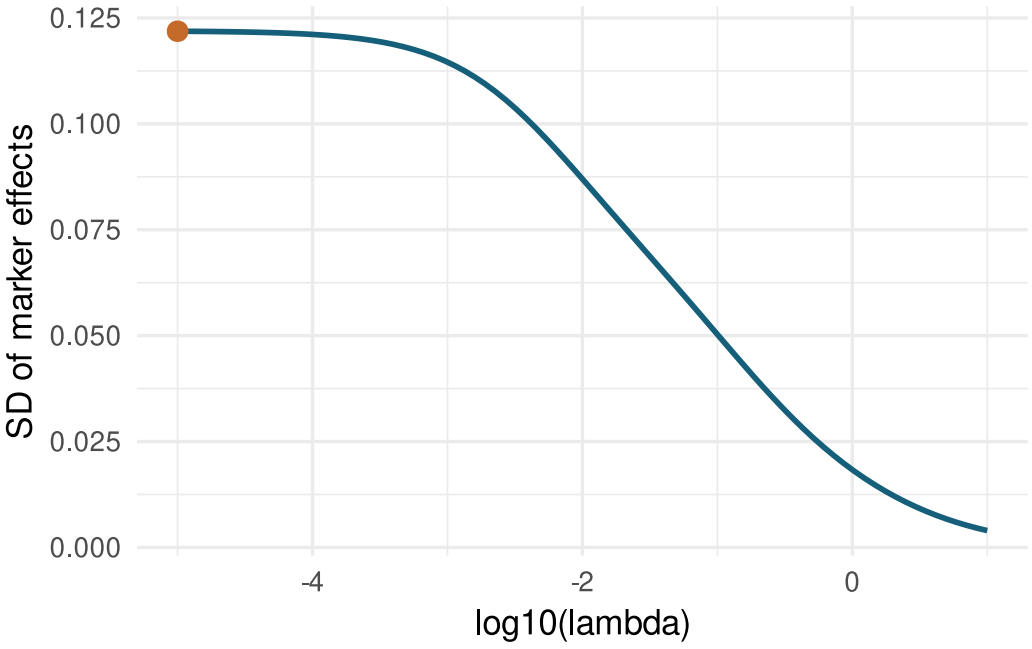
lambda	mse	penalty	criteria	sd_marker_effect
0	0.0116	0	0.0116	0.1219

```
ggplot(ridge_path, aes(x = log10(lambda), y = criteria)) +
  geom_line(color = "#155f7a", linewidth = 1) +
  geom_point(data = best_lambda, color = "#c46a2b", size = 3) +
  labs(x = "log10(lambda)", y = "Ridge criterion")
```



As λ increases, marker effects are shrunk more strongly toward zero.

```
ggplot(ridge_path, aes(x = log10(lambda), y = sd_marker_effect)) +
  geom_line(color = "#155f7a", linewidth = 1) +
  geom_point(data = best_lambda, color = "#c46a2b", size = 3) +
  labs(x = "log10(lambda)", y = "SD of marker effects")
```




```
library(BGLR)
data(wheat)

subset = 1:100

y = wheat.Y[subset,2]

M = wheat.X

# take a subset of columns
M = M[subset,1:300]

# design matrix for fixed effects (no shrinkage)
X = matrix(data = 1, nrow = length(y), ncol = 1)
colnames(X) = "intercept"

# we can use the same function as before for finding the optimum feature weights
mse_ridge <- function(par, y, M, X) {

  # the weights are stored in "par", the length of that vector depends
  # on the number of columns in X (number of features)

  # get expectation from fixed effects
  b = par[1:ncol(X)]
```

```
  mu = X %*% b

  # predicted value
  u = par[(ncol(X) + 1):(length(par) - 1)]
  pred = mu + M %*% u

  # mean squared error
  mse = mean((y - pred)**2)

  ridge_param = par[length(par)]

  # ridge penalty
  ridge = ridge_param * sum(u**2)

  criteria = mse + ridge

  # force ridge parameter to be between  > 0
  if(ridge_param < 0) {
    criteria = 9999999
  }

  return(criteria)

}

# initialize
p <- c(rep(0, ncol(X) + ncol(M)), runif(1))

# give names for p
names(p) <- c(colnames(X), colnames(M), "lambda")

res <- optim(
       par = p,
       fn = mse_ridge,
       y = y,
       X = X,
     M = M,
     control = list(maxit = 20000),
     method = c(
      #"Nelder-Mead"
      #"BFGS"
      #"CG"
      "L-BFGS-B"
```

```
      #"SANN"
      #"Brent"
            )
)

# our solution
res$par

print(paste("Ridge Parameter:", res$par[length(res$par)]))

# plot predicted vs observed
# get predicted values
mu = X %*% res$par[1:ncol(X)]
pred = mu + M %*% res$par[(ncol(X) + 1):(length(res$par) - 1)]

dat = data.frame(
     y = y,
     pred = pred,
     id = 1:length(y)
     )

ggplot(dat, aes(x = pred, y = y)) +
 geom_point()

## plot raw data and insert intercept and slope
#ggplot(dat, aes(x = Temp, y = Wind)) +
#  geom_point() +
#  geom_abline(intercept = res$par[1], slope = res$par[2], color = "red")
```

Now we compare the results from the optimization to an equivalent mixed model approach with a single randome effect:

$$\begin{bmatrix} X'X & X'Z \\ Z'X & Z'Z + I\lambda \end{bmatrix} \begin{bmatrix} \beta \\ u \end{bmatrix} = \begin{bmatrix} X'y \\ Z'y \end{bmatrix}$$

```
library(rrBLUP)
mod_rr = mixed.solve(y = y, X = X, Z = M)

# ridge parameter from MME
ridge_mme = mod_rr$Ve / mod_rr$Vu
# preditions from that model
pred_mme = X %*% mod_rr$beta + M %*% mod_rr$u
print(ridge_mme)
```

```
dat = data.frame(
     y = pred,
     pred = pred_mme,
     id = 1:length(y)
     )

ggplot(dat, aes(x = pred, y = y)) +
 geom_point()
```

Advanced Models

Mixed Model